

Track B Stage-B Head — Multi-Task Labels Overview

This note documents the changes made to the Track B (Stage-B head) pipeline so that each **candidate ROI** carries explicit multi-task supervision, while keeping the existing single-task (next-active + TTC regression) behavior intact.

The goal is to make it easy to plug in future **noun / verb / TTC-bin heads** without touching the dataset again.

1. What Changed

1.1 Dataset (`TrackBDataset`)

File: `local_extraction/trackB/trackB_dataset.py`

The dataset now:

- Still supports generic manifests (`candidates` / `boxes` / `objects` with `cls` and `ttc`).
- Adds explicit per-candidate fields for multi-task STA:
 - `is_positive` — next-active flag (0/1).
 - `gt_noun_id` — noun category ID (int, `-1` if unknown).
 - `gt_verb_id` — verb category ID (int, `-1` if unknown).
 - `gt_ttc` — TTC in seconds (float).
 - `gt_ttc_bin` — integer TTC bin index.

The Stage B head manifests (`head_train.jsonl`, `head_val.jsonl`) already contained:

- `verb_id`, `noun_id`, `ttc`, `is_positive`.

`_maybe_convert_stageB_records()` now preserves these fields when it groups the records by (`uid`, `frame`).

`_parse_record()` converts each raw candidate into a normalized dict:

- Always keeps:
 - `bbox` — (x1, y1, x2, y2).
 - `cls` — generic class label (kept for backward compatibility).
 - `ttc` — TTC in seconds.
- Newly adds:
 - `is_positive` — derived from explicit `is_positive` when present, else from `cls` (class 1 → positive).
 - `verb_id` / `noun_id`.
 - `ttc_bin` — computed via `ttc_to_bin(ttc)`.

`__getitem__()` now returns, per sample (per image with a variable number of candidates):

```
{
  "uid": uid,
  "frame_path": frame_path,
  "img_tokens": img_tokens,      # (N, 512)
  "vid_tokens": vid_tokens,      # (T, N, 512)
  "hw": hw,                      # token grid size
  "image_size": (W, H),
  "bboxes": [...],              # list of candidate boxes

  # Classification labels
  "labels": LongTensor[Nc],      # legacy cls (currently same as is_positive for
Stage B heads)
  "is_positive": LongTensor[Nc], # explicit next-active flag 0/1

  # Semantic labels
  "gt_noun_id": LongTensor[Nc],  # noun ID or -1
  "gt_verb_id": LongTensor[Nc], # verb ID or -1

  # TTC labels
  "gt_ttc": FloatTensor[Nc],     # TTC in seconds
  "gt_ttc_bin": LongTensor[Nc],  # TTC bin index
  "ttc": FloatTensor[Nc],        # legacy alias for gt_ttc
  "ttc_norm": FloatTensor[Nc],   # normalized TTC (z-score)

  "valid": True,
}
```

1.2 TTC Bins

A simple binning helper was added:

- Constant thresholds:

```
TTC_BIN_THRESHOLDS = (0.5, 1.0, 2.0)
```

- Helper:

```
def ttc_to_bin(ttc: float, thresholds=TTC_BIN_THRESHOLDS) -> int:
    # 0: [0.0, 0.5), 1: [0.5, 1.0),
    # 2: [1.0, 2.0), 3: [2.0, +inf)
```

This is used inside the dataset to populate `ttc_bin` / `gt_ttc_bin` for each candidate.

2. Head Architecture and Training Loss (Now Multi-Task)

2.1 Head (`TrackBHead`)

File: `local_extraction/trackB/trackB_head.py`

The head now supports **multi-task outputs** per candidate ROI:

- `cls_logits` — next-active vs other (K classes, typically 2).
- `ttc` — continuous TTC regression (seconds, normalized in training).
- `noun_logits` — optional noun classification head, shape `(B, Nc, num_noun_classes)`.
- `verb_logits` — optional verb classification head, shape `(B, Nc, num_verb_classes)`.
- `ttc_bin_logits` — optional TTC-bin head, shape `(B, Nc, num_ttc_bins)`.

These are controlled via `HeadConfig`:

- `num_noun_classes`, `num_verb_classes`, `num_ttc_bins` (default 0 → head disabled).
- `ttc_mode`: "reg" (regression, default) or "bin" (classification).

`TrackB_train_loader.py` infers `num_noun_classes`, `num_verb_classes`, and `num_ttc_bins` from the Stage B manifests (max ID + 1), gated by:

- `TrainConfig.use_multi_task_labels`
- `TrainConfig.use_ttc_bins`

2.2 Main training loop and multi-task loss

File: `local_extraction/trackB/trackB_train_loader.py`

The main training path (`mode='main'`) now:

- Builds features exactly as before (ROI-pooled fused tokens).
- Consumes the new fields from the dataset:
 - `is_positive` → next-active classification target.
 - `gt_noun_id`, `gt_verb_id` → noun/verb targets.
 - `gt_ttc`, `gt_ttc_bin` → TTC regression / bin targets.
- Defines a weighted multi-task loss:

```
loss = (
    w_next * L_next          # CE on is_positive
  + w_ttc * L_ttc           # SmoothL1 on TTC or CE on gt_ttc_bin
  + w_noun * L_noun         # CE on gt_noun_id (positive candidates)
  + w_verb * L_verb         # CE on gt_verb_id (positive candidates)
)
```

where weights come from `TrainConfig`:

- `loss_w_next`, `loss_w_noun`, `loss_w_verb`, `loss_w_ttc` (all default 1.0).

Behavior by mode:

- If `use_multi_task_labels=False` and `use_ttc_bins=False` (default):
 - Only `L_next` (next-active) and continuous `L_ttc` are active → **exactly the old behavior**.
- If `use_multi_task_labels=True`:

- Noun/verb heads are enabled; `L_noun` and `L_verb` are trained on candidates with `is_positive==1`.
- If `use_ttc_bins=True`:
 - TTC classification loss `L_ttc` is computed from `ttc_bin_logits` vs `gt_ttc_bin` instead of regression; the dataset still exposes continuous `gt_ttc` if you want to combine both later.

The training loop logs:

- Per-step: total loss, `L_next`, `L_ttc`.
- Per-epoch: averages for total loss, `next`, `ttc`, `noun`, `verb`.

2.3 Demo / validation / pruning

- Loader demo variant (`_run_demo`) uses `is_positive` as the classification label when available.
- Validation helper `_evaluate_loader()` uses:

```
labels = s.get("is_positive", s["labels"]).cpu()
```

- `trackB_eval.py` and `trackC_pruning.py` do the same when assembling labels, keeping all evaluation code aligned with the explicit next-active flag.

3. N / N+V / N+ δ Evaluation (Multi-Task Metrics)

File: `local_extraction/trackB/trackB_eval.py`

The evaluation script still computes:

- Candidate-level accuracy, mAP, and TTC MAE (as before).

In addition, when the checkpoint head exposes `noun_logits` and `verb_logits`, it now computes **frame-level** metrics:

- **N_mAP** — fraction of frames where:
 - The predicted top candidate (highest `p(next_active)`) has `IoU` $\geq$ `iou_thresh` with the positive GT box, and
 - `pred_noun_id == gt_noun_id`.
- **Nv_mAP** — fraction of frames where:
 - `IoU` $\geq$ `iou_thresh`, and
 - Both `pred_noun_id == gt_noun_id` and `pred_verb_id == gt_verb_id`.
- **N_delta_mAP** — fraction of frames where:
 - `IoU` $\geq$ `iou_thresh`,
 - `pred_noun_id == gt_noun_id`, and
 - TTC bin matches:
 - Either `ttc_bin_logits` argmax (if `ttc_mode='binned'`),
 - Or `ttc_to_bin(pred_ttc_seconds)` (if `ttc_mode='reg'`).

Implementation details:

- For each frame, the evaluator:
 - Finds the **GT positive candidate** using `is_positive`.
 - Uses its `gt_noun_id`, `gt_verb_id`, `gt_ttc_bin` as ground truth.
 - Picks the predicted candidate with max `p(next_active)` and compares boxes via IoU.
- IoU threshold and TTC mode are configurable via `EvalConfig` / CLI:
 - `iou_thresh` (default `0.5`),
 - `ttc_mode` in `{ "reg", "binned" }`.
- Results are added to the metrics JSON as:
 - `N_mAP`, `Nv_mAP`, `N_delta_mAP` (plus existing `ttc_mae_seconds`).

CLI usage example:

```
python local_extraction/trackB/trackB_eval.py \
  --checkpoint local_extraction/runs/Track_B/checkpoints/trackB_best.pt \
  --ttc_mode reg
```

This writes `metrics_val_*.json` under `local_extraction/runs/Track_B/metrics/` with both the original metrics and the multi-task N / N+V / N+ δ values when the model has the corresponding heads.

4. Single-Task vs Multi-Task Summary

- **Single-task Track B (baseline):**
 - Enable: keep `use_multi_task_labels=False`, `use_ttc_bins=False`.
 - Head outputs: `cls_logits`, `ttc`.
 - Loss: `L_next` + `L_ttc` only.
 - Metrics: accuracy, mAP, candidate-level TTC MAE (no N/N+V/N+ δ).
- **Multi-task Track B (noun + verb + TTC-bin):**
 - Enable: `use_multi_task_labels=True`, optionally `use_ttc_bins=True`.
 - Head outputs: `cls_logits`, `ttc`, and (optionally) `noun_logits`, `verb_logits`, `ttc_bin_logits`.
 - Loss: weighted combination of `L_next`, `L_noun`, `L_verb`, `L_ttc`.
 - Metrics: original metrics + `N_mAP`, `Nv_mAP`, `N_delta_mAP`.

This makes it straightforward to align implementation with the thesis-level metrics **N**, **N+V**, and **N+ δ** while preserving the original baseline behavior as a special case.

5. Why This File Exists

This markdown file serves three purposes:

1. **Documentation of label plumbing.** It explains how Stage B head manifests (`head_train.jsonl` / `head_val.jsonl`) are converted into per-candidate labels and how they surface in `TrackBDataset`.
2. **Clarification of semantics.** It disambiguates the roles of:
 - `cls` / `labels` (generic label, currently aligned with `is_positive` for Stage B runs),
 - `is_positive` (next-active flag),

- `gt_noun_id, gt_verb_id, gt_ttc, gt_ttc_bin`.

3. **Design note for future multi-task heads.** It records where to plug in noun/verb/TTC-bin heads and how to control them via config toggles, so future thesis iterations or collaborators can extend the model without re-reverse-engineering the data pipeline.

In short: this file is the **contract** for multi-task labels and heads in Track B, ensuring the Stage B head can grow from a purely binary + TTC model into a full `{next-active, noun, verb, TTC}` predictor with N / N+V / N+ δ metrics, all with minimal code churn.